



Inżynieria Oprogramowania

dr Katarzyna Grzesiak-Kopeć

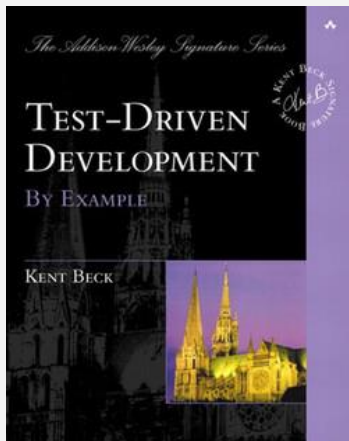
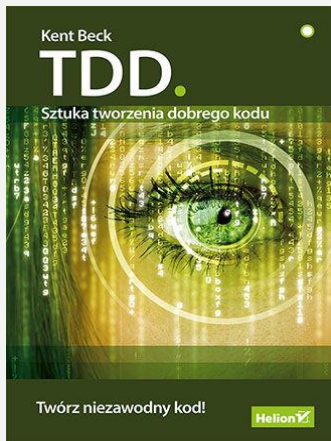




10 TDD & CI/CD DevOps



Pomysłodawca



2002



Kent Beck

Chrysler C3 project, 1997

Programowanie poprzez testy (TDD)
Ciągła integracja (Continuous Integration, CI)

Testowanie projektów OO



Obiekty

Weryfikacja zachowania i współpracy z innymi obiektami

Sygnały

Nadawca, odbiorca, parametry

Klasa

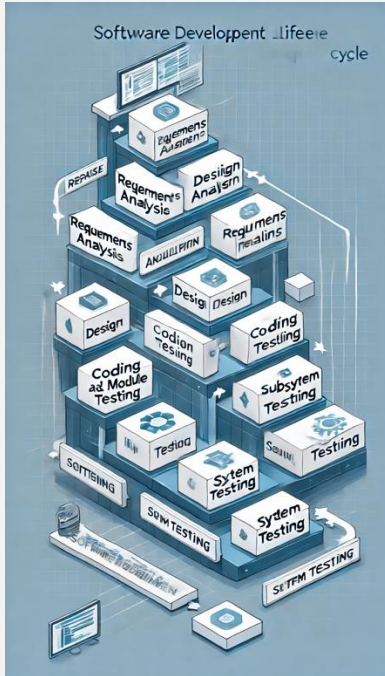
Specyfikacja klasy

Implementacja klasy

Dziedziczenie

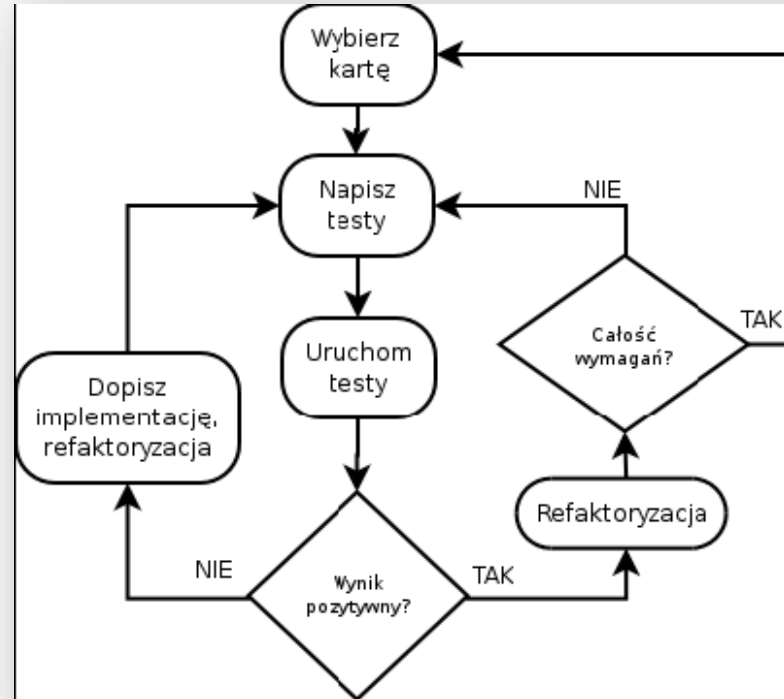
Polimorfizm

Testy w technice wodospadu



- Odsuwa w czasie integrację i agreguje testowanie
- Opóźnienia w implementacji – mniej czasu na testy
- Przekroczony budżet – obcinamy testy
- Zespół QA (Quality Assurance)
Programiści vs Testerzy

Test Driven Development



Zalety TDD

-  **Feedback:** natychmiastowa informacja zwrotna
-  **Task-orientation:** lepsza dekompozycja realizacji projektu
-  **Quality assurance:** aktualne testy – określony poziom jakości kodu
-  **Low-level design:** lepsza dekompozycja problemu
-  Wyższa jakość i produktywność



Krytyka TDD



Tylko dla doświadczonych programistów

Pisanie testów do nieistniejącej implementacji



Wymaga dużej samodyscypliny

Najpierw test potem funkcjonalność



Wydłuża proces tworzenia oprogramowania

Dużo czasu na testy



Continuous Integration (CI)



- Testowanie każdej zmiany w systemie
- Dedykowane narzędzia zintegrowane z:
 - Repozytorium kodu źródłowego
 - Systemem raportowym
- Natychmiastowe wykrycie błędów

Praktyki CI

- 01 Jedno repozytorium kodu
- 02 Automatyzacja budowy
- 03 Testowanie to część budowy
- 04 Codzienna aktualizacja źródeł
- 05 Zmiana w repozytorium wyzwala integrację
- 06 Szybka budowa aplikacji (~10min)
- 07 Środowisko testowe klonem produkcyjnego (staging)
- 08 Wszyscy widzą co się dzieje



• Zalety CI •



Zmniejszenie ryzyka



Nie odsuwa w czasie integracji i nie agreguje testowania



Ułatwia znalezienie i usunięcie błędów



Umożliwia szybkie wdrażanie



Platforma programistyczna

Adam Perlik, Środowisko ciągłej integracji do budowy aplikacji zgodnych z J2EE, 2009



Środowisko programistyczne:

Apache Maven, Subversion, Eclipse IDE



Biblioteki programistyczne:

Springframework, JUnit, Mockito



Narzędzia testujące:

Selenium Remote Control, Soap UI



Serwer ciągłej integracji:

Hudson (Jenkins)



Serwer natychmiastowej komunikacji

Openfire Server

Platforma programistyczna Java 2024



Środowisko programistyczne:

IntelliJ IDEA z Gradle jako system budowania



Biblioteki programistyczne:

JUnit, Mockito, AssertJ



Narzędzia testujące:

wbudowana obsługa JUnit w IntelliJ IDEA, JaCoCo



Serwer ciągłej integracji:

GitHub Actions, Jenkins lub GitLab CI.

Platforma programistyczna Python 2024



Środowisko programistyczne:

PyCharm z pytest



Biblioteki programistyczne:

pytest, pytest-mock, tox, coverage.py



Narzędzia testujące:

wbudowana obsługa pytest w PyCharm, flake8



Serwer ciągłej integracji:

GitHub Actions, Travis CI lub GitLab CI.

Dlaczego GitHub?



GitHub

Integracja z CI/CD:

- wbudowane narzędzie GitHub Actions

Popularność

Łatwa integracja

- JUnit, pytest, SonarQube (analiza kodu)

Dlaczego GitHub Actions?



Integracja z GitHub:

- działa natywnie w ramach GitHub
- automatyczne uruchamianie *workflow* na podstawie zdarzeń

Łatwa konfiguracja

- konfiguracja w plikach YAML
- liczne gotowe szablony *workflow*

Wszechstronność i elastyczność

- szeroka gama akcji (budowanie, testowanie, wdrażanie, analiza kodu itd..)
- dowolny język (Node.js, Python, Java, Ruby, PHP, Go, Rust, .NET...)
- dowolna platforma (Linux, macOS, Windows, ARM, GPU)

CI/CD



**Continuous
Integration**



**Continuous
Delivery**



**Continuous
Deployment**

CI/CD



Continuous Integration

Cel: automatyczna integracja kodu w centralnym repozytorium

Koniec: działający artefakt (np. binarka) gotowy do wdrożenia lub dalszego testowania

Przykład

- Programista wysyła kod do repozytorium
- System CI automatycznie buduje projekt i uruchamia testy
- Gdy są błędy proces zatrzymuje się, a programista dostaje informację zwrotną

CI/CD



Continuous Delivery

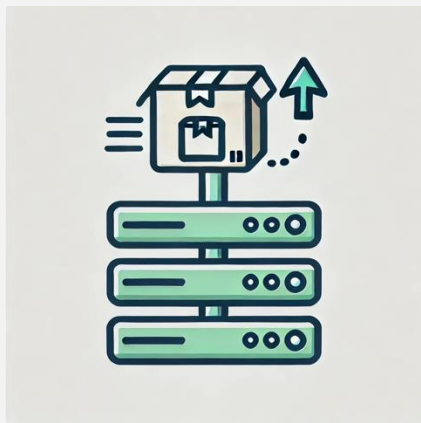
Cel: automatyczne przygotowanie kodu do wdrożenia w środowisku produkcyjnym; rozszerza CI (testy na wszystkich poziomach)

Koniec: działający kod gotowy do wdrożenia

Przykład

- Po CI kod automatycznie przekazany na *staging*
- Zespół QA przeprowadza dodatkowe testy przed ręcznym wdrożeniem na produkcję

CI/CD



Continuous Deployment

Cel: automatyczna wdrożenie każdej zmiany, która przeszła testy na środowisko produkcyjne

Koniec: całkowicie automatyczne wdrożenie

Przykład

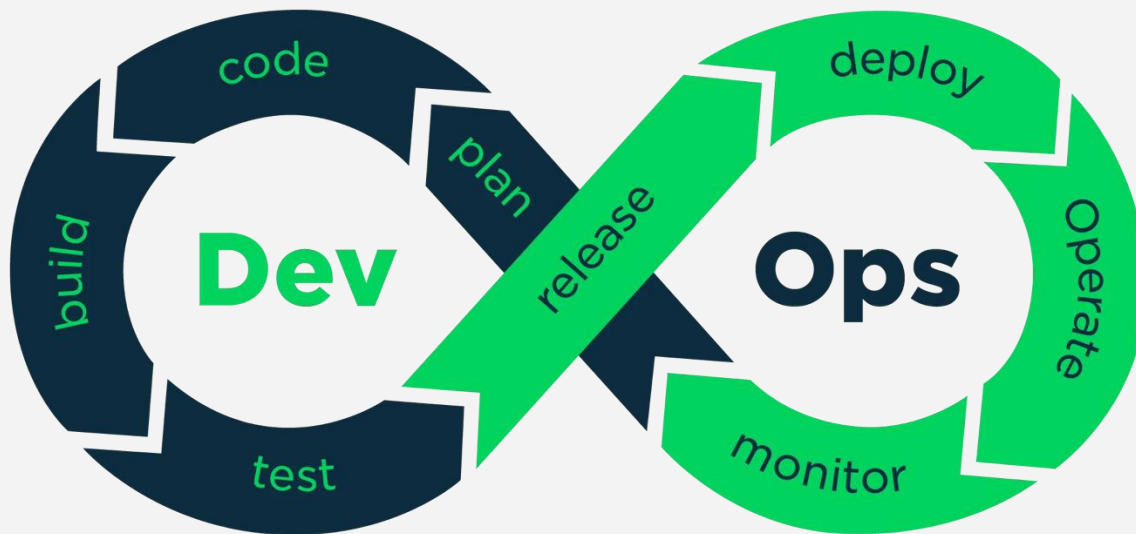
- Po zakończeniu testów integracyjnych i akceptacyjnych system automatycznie wdraża kod na serwer produkcyjny

Porównanie



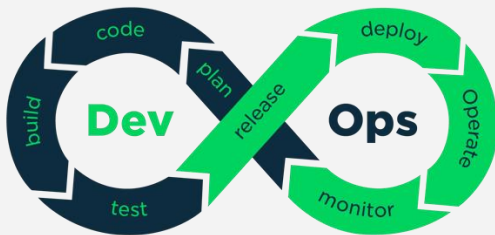
	Continuous Integration (CI)	Continuous Delivery (CD)	Continuous Deployment (CD)
Cel	Integracja kodu i automatyczne testowanie	Automatyczne przygotowanie do wdrożenia	Automatyczne wdrożenie na produkcję
Koniec procesu	Po zakończeniu testów	Kod gotowy do ręcznego wdrożenia	Automatyczne wdrożenie na środowisko produkcyjne
Zakres automatyzacji	Testy jednostkowe, integracyjne	Automatyzacja testów i przygotowania wdrożenia	Pełna automatyzacja od testów do produkcji
Ręczna interwencja	Niezbędna, by zbudowany kod przeszedł do dalszych etapów	Potrzebna przy wdrażaniu na produkcję	Brak (wszystko dzieje się automatycznie)

DevOps = Development + Operations



Andrew Shafer and Patrick Debois first introduced the term publicly in 2008 at the Agile Toronto conference.

DevOps = Development + Operations



- Zestaw praktyk, metodologii i narzędzi
- Cel to usprawnienie współpracy między:
Deweloperami (Development)
Administratorami IT (Operations)
- W procesie tworzenia, testowania, wdrażania i zarządzania oprogramowaniem



Koniec!